# VERIFICATION TEST PLAN

ECE 506: Project

Maseeh College of Engineering and Computer Science

Fall, 2026




**Project Name:** Formal Verification for an I²C-Based Memory Subsystem

**Members:** Thanmai Neelampalli, Chitra Vagdevi Shivani Kottala, Bhanu Sai Sidhardha Tummala

**Document:** Checkpoint 1 — Verification Plan and Initial Implementation

**Date:** 10/22/2026

# 1 Table of Contents


**2 Introduction:** ..... 2

**2.1 Objective of the verification plan** ..... 3

**2.2 Top Level block diagram** ..... 3

**3 Verification Requirements** ..... 4

**3.1 Verification Levels** ..... 4

**4 Required Tools** ..... 6

**4.1 Required Tools and Software** ..... 6

**5 Risks and Dependencies** ..... 6

**5.1 Critical threats** ..... 6

**5.2 Contingency and Mitigation plans.** ..... **Error! Bookmark not defined.**

**6 Functions to be Verified.** ..... 7

**6.1 Functions from specification and implementation** ..... 7

**6.2 Critical and Non-Critical Functions** ..... 7

**7 Tests and Methods** ..... 10

**7.1.1 Black-Box, White-Box, and Gray-Box Testing.** ..... 10

**7.1.2 Advantages and Disadvantages of Testing Methods** ..... 11

**7.1.3 Verification Environment Architecture** ..... 11

**7.1.4 Verification Strategy.** ..... 12

**7.1.5 Property Development and Formal Stimulus** ..... 12

**7.1.6 Checking Methodology** ..... 13

**7.1.7 Test-Case Scenarios Matrix** ..... 14

**8 Coverage Requirements** ..... 15

**8.1.2 Assertions and Property Coverage** ..... 15

**9 Checkpoint Deliverables and Project Schedule** ..... **Error! Bookmark not defined.**


# 2 Introduction:

The I²C-based memory subsystem is a digital system designed to perform memory read and write operations through an I²C-style serial communication interface. The design consists of

four major functional blocks: a Functional Unit (FU), an I²C Master, an I²C Slave/Memory Controller, and a Memory Block. The Functional Unit is implemented as an 8-bit universal shift register that supports load, hold, shift, rotate, and arithmetic-shift operations. The I²C Master controls the transaction sequence and generates the necessary START, STOP, address, data, and acknowledgment phases. The I²C Slave receives serial commands, interprets the address and read/write information, and coordinates memory access. The design also includes address-range checking, repeated-START handling, multi-byte transfers, and clock-stretching support. The verification effort focuses on determining whether the implemented RTL behaves according to the project's intended functional requirements. This verification plan defines the properties, assumptions, formal-analysis procedures, test scenarios, and coverage goals for the project. The work is divided into three checkpoints to provide measurable progress from initial verification planning to comprehensive analysis and final documentation.

## 2.1 Objective of the verification plan

The primary objective is to verify the functional correctness and protocol behavior of the I²C-based memory subsystem using SystemVerilog Assertions (SVA) and formal verification techniques.

The verification plan aims to:

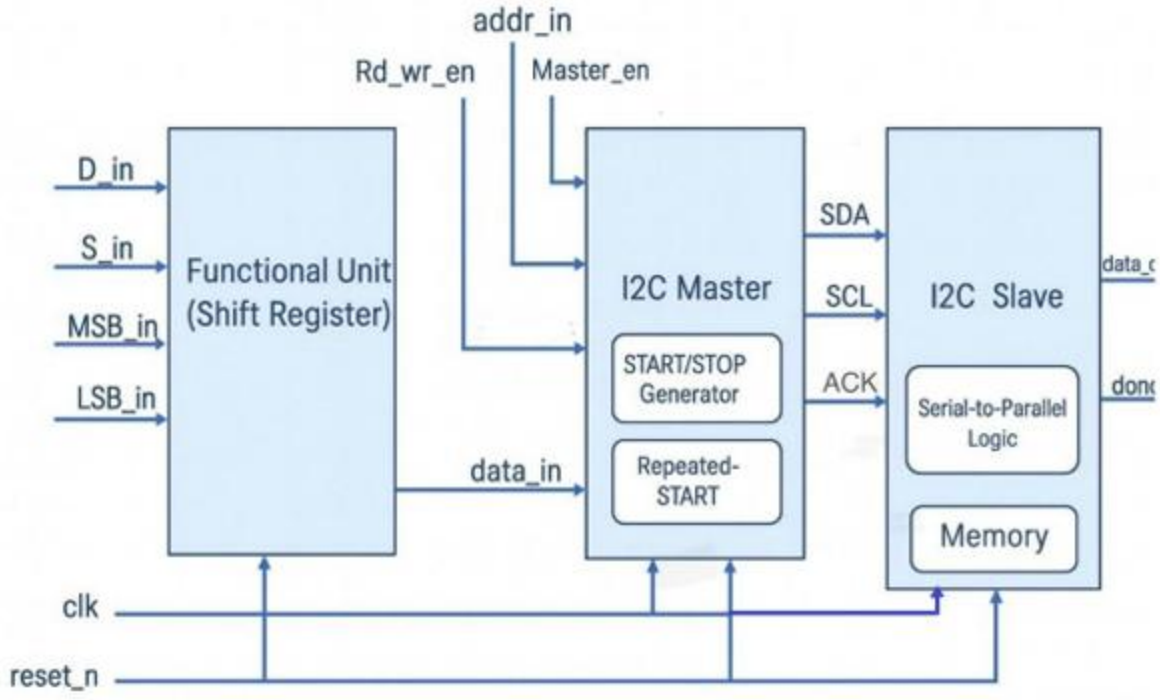
- Verify the Functional Unit's supported data-processing operations.
- Validate the I²C Master and Slave finite state machines.
- Check the generation and detection of START and STOP conditions.
- Verify address transmission, R/W decoding, and ACK/NACK behavior.
- Confirm that valid write transactions update the intended memory locations.
- Confirm that read transactions return the previously stored data.
- Verify that illegal addresses do not cause unintended memory updates.
- Validate repeated-START sequences, multi-byte transfers, and clock stretching.
- Develop appropriate assumptions to model valid operating conditions for formal analysis.
- Use assertion results, formal counterexamples, and waveforms to investigate failures.
- Measure verification progress through property results and applicable coverage metrics.

The final objective is to establish evidence that the subsystem meets its defined requirements under the operating conditions represented by the verification environment.

## 2.2 Top Level block diagram

The subsystem contains the following major blocks:

1. Functional Unit
2. I²C Master
3. I²C Slave/Memory Controller
4. Memory Block




**Block Diagram**

The block diagram should identify the data and control connections between the Functional Unit, Master, Slave, and Memory. It should also show the SDA and SCL communication signals and the relevant control/status signals, such as Master_en, ACK, data_out, and done, wherever applicable to the RTL interfaces.

# 3 Verification Requirements

The verification process will examine the subsystem at multiple levels, starting with individual modules and progressing toward the integrated top-level design.

## 3.1 Verification Levels

### 3.1.1 Module – Level Verification

Individual modules will be analyzed to verify their functional behavior independently.

The primary targets are:

- D flip-flop
- 8-to-1 multiplexer
- Functional Unit / universal shift register
- I²C Master
- I²C Slave/Memory Controller

- Memory logic

For the D flip-flop and multiplexer, sequential or combinational equivalence checks will be selected according to the implementation and reference model available. The Functional Unit will be checked against its specified operation for each supported control input.

### 3.1.2 Protocol – Level Verification

Protocol-level verification will focus on the behavior of SDA and SCL and the sequencing of $I^2C$ transactions.

**The properties will examine:**

- START and STOP conditions.
- Address and R/W bit sequencing.
- Data-bit transmission and reception.
- ACK/NACK response behavior.
- SDA stability during SCL-high periods, allowing START and STOP exceptions.
- Repeated-START behavior.
- Multi-byte transaction sequencing.

### 3.1.3 Subsystem – Level Verification

The integrated subsystem will be checked to ensure that the Functional Unit, Master, Slave, and Memory work together correctly. The main objective is to verify that data is processed, transmitted, stored, and retrieved as expected. Checks will include correct coordination between the master and slave state machines, valid memory accesses, and readback consistency.

### 3.1.4 Formal Verification and Equivalence Checking

Formal Property Verification (FPV) will be used to determine whether specified properties hold for the modeled design and environment.

Sequential equivalence checking will be applied to suitable modules when a valid reference implementation and corresponding comparison setup are available.

Formal failures will be analyzed using counterexamples to identify whether the cause is an RTL defect, an incorrect assertion, or an unsuitable environmental assumption.

### 3.1.5 Regression and Integration Verification

As the assertion files and RTL evolve, previously passing properties will be rerun to detect regressions. The integrated design will be analyzed after the individual modules have been checked sufficiently to support subsystem-level verification.

# 4 Required Tools

## 4.1 Required Tools and Software

### 4.1.1 Synopsys VC Formal

VC Formal will be used for formal property verification and supported sequential equivalence checks.

The environment will include the appropriate application mode, RTL file lists, assertion files, clock/reset definitions, and property assumptions.

### 4.1.2 SystemVerilog

SystemVerilog will be used to implement the RTL and verification properties. SVA constructs will express temporal relationships between protocol signals, state transitions, control signals, and data values.

### 4.1.3 RTL and SVA Source Files

The project source files will include the subsystem RTL, module-level assertion files, top-level verification properties, and the relevant scripts and file lists.

### 4.1.3 Waveform and Log Analysis

Waveforms, formal logs, and counterexample traces will be examined to understand property failures and verify the sequence of events leading to each result.

### 4.1.3 Version Control and Documentation

The source files, verification scripts, test plan, and results will be maintained in an organized project repository. Actual repository details will be included in the final submission.

# 5 Risks and Dependencies

## 5.1 Critical threats

**Master–Slave Synchronization Issues**

The master and slave use related but distinct state machines to control and interpret serial transactions. Incorrect assumptions about clock edges or state-transition timing may cause unexpected assertion failures.

**Protocol Assertion Errors**

Assertions that do not account for legal START/STOP transitions, ACK timing, or clock stretching may report false failures.

**Incorrect Environmental Assumptions**

Overly restrictive assumptions may prevent the formal engine from exploring important failure scenarios. Insufficient assumptions may permit unrealistic input sequences that complicate analysis.

**Illegal Address Handling**

The design distinguishes legal addresses 0–127 from out-of-range addresses 128–255. Assertions must check the intended response for each category and ensure that illegal addresses do not result in unintended writes.

**End-to-End Data Consistency**

A write-read property may fail if its antecedent and consequent do not refer to the same address and transaction, or if the formal environment permits unrelated intervening operations.

**Compilation and Elaboration Problems**

Mismatched signal widths, inconsistent state names, missing source files, or incorrect assertion bindings may prevent successful formal analysis.

**Contingency and Mitigation Plans**

**The following measures will be used to reduce these risks:**

- Review the RTL state machines and timing behavior before writing temporal assertions.
- Begin with simple module-level properties before introducing integrated transaction properties.
- Validate assertion syntax, signal widths, and hierarchy references during elaboration.
- Introduce assumptions only when supported by the intended interface behavior.
- Inspect counterexamples before changing the RTL or relaxing an assertion.
- Separate genuine design failures from incorrect property expectations.
- Use directed checks for critical protocol conditions and add broader properties incrementally.
- Maintain a record of unresolved failures and rerun previously passing properties after changes.

# 6 Functions to be Verified.

## 6.1 Functions from specification and RTL implementation

**D Flip-Flop**

- The D flip-flop will be checked for correct register behavior.
- Properties include:
- The output follows the input at the appropriate active clock edge.
- An asynchronous clear, if present in the RTL, forces the output to the specified reset value.
- The output holds its value when no update is required.

**8-to-1 Multiplexer**

- The multiplexer will be checked for correct selection behavior.
- Properties include:
- Each select value routes the corresponding input to the output.
- The output matches the selected input under the specified combinational semantics.
- The implementation matches the reference behavior when equivalence checking is applicable.

**Functional Unit**

- The Functional Unit is an 8-bit universal shift register controlled by a 3-bit select input.
- The following functions will be verified:
- Hold / NOP.
- Parallel load.
- Logical shift left and right.
- Rotate operations.
- Arithmetic-shift operations.
- Correct use of MSBIn and LSBIn during applicable shift operations.
- Correct output for every supported control encoding.

**I²C Master**

- The I²C Master is responsible for controlling the transaction sequence.
- The following functions will be verified:
- Idle behavior and transaction initiation.
- START condition generation.
- Address and R/W bit transmission.
- Data transmission and reception control.
- ACK/NACK sampling and response handling.
- STOP condition generation.
- Bit-counter and burst-counter behavior.
- Multi-byte transfers.
- Repeated-START operation.
- Proper state transitions during normal and exceptional transaction sequences.

**I²C Slave/Memory Controller**

- The slave receives commands and controls memory access.
- The following functions will be verified:
- START and STOP detection.
- Address and R/W bit collection.
- Serial-to-parallel conversion of received data.

- Legal and illegal address handling.
- Appropriate ACK/NACK behavior.
- Memory write control.
- Read-data availability and done signaling.
- Clock-stretching behavior through scl_stretch_n.
- Correct state transitions and bit-counter operation.

**Memory Block**

- The Memory Block will be checked for:
- Correct data storage on a valid write.
- Correct data retrieval from the requested address.
- Preservation of stored contents during read operations.
- Prevention of unintended writes on invalid accesses.
- Correct interaction with the slave controller.

**Integrated Subsystem**

- The top-level verification will check:
- Correct communication between the master and slave.
- Correct handling of valid read and write transactions.
- End-to-end data consistency.
- Appropriate responses to invalid addresses.
- Multi-byte transfer sequencing.
- Repeated-START behavior.
- Correct operation when clock stretching is exercised.

### 4.1.1 Critical and Non-Critical Functions

**Critical functions** are those that directly affect data integrity, transaction correctness, and safe memory access.

These include:

- Correct address and data transfer.
- Correct ACK/NACK behavior.
- Correct memory read and write operations.
- Correct START/STOP sequencing.
- Correct handling of illegal addresses.
- Correct state-machine operation.
- Correct end-to-end data consistency.

**Additional functional scenarios** include repeated START, multi-byte transfers, clock stretching, and less common control combinations. These are included in the verification plan because they are part of the stated project functionality.

No feature will be excluded from verification solely because it is difficult to analyze. Any scope limitation will be documented with its technical justification.

# 7 Tests and Methods

## 7.1.1 Testing methods to be used: Black/White/Gray Box.

**Black-Box Testing**

Black-box testing evaluates the subsystem through its specified inputs and outputs without relying on internal implementation details.

**Application to this project:**

- Verify valid read and write transactions.
- Check returned data against previously written data.
- Check the response to legal and illegal addresses.
- Verify transaction completion and external protocol behavior.

**White-Box Testing**

White-box testing uses knowledge of the RTL implementation to verify internal logic and state behavior.

**Application to this project:**

- Check master and slave state-machine transitions.
- Verify address and data counters.
- Check shift-register operations.
- Verify memory write-enable behavior.
- Examine internal control signals and state transitions.

**Gray-Box Testing**

Gray-box testing combines external behavior checks with selected knowledge of internal implementation.

**Application to this project:**

- Relate external START/STOP conditions to internal FSM transitions.
- Check the relationship between received addresses and memory control.
- Investigate ACK/NACK failures using internal state and counter information.
- Relate clock stretching to master–slave transaction progress.

### 7.1.2 Advantage and Disadvantage of testing methods.

| Method | Advantages | Disadvantages | Application |
|---|---|---|---|
| Black-box | Validates externally visible behavior and interface requirements | May provide limited insight into the source of failures | Transaction correctness and read/write behavior |
| White-box | Enables detailed checks of internal control and data paths | Requires RTL understanding and correct hierarchy references | FSMs, counters, shift register, and memory control |
| Gray-box | Combines interface checks with internal debugging information | Requires coordination between external and internal observations | Protocol sequencing and master–slave integration |

All three methods are useful. Formal verification will be the primary analysis method for the properties defined in this project, supported by RTL inspection and waveform analysis.

### 7.1.3 Verification Enviornment Architecture

The verification environment will be organized around the RTL design, SVA properties, formal assumptions, and analysis scripts.

**Design Under Verification (DUV)**

The DUV is the $I^2C$-based memory subsystem, either at an individual module level or at the integrated top level.

**Assertion Modules**

Assertion modules will contain properties for the applicable design functions. These may be bound to internal modules or instantiated in the verification hierarchy, depending on the available RTL structure.

**Formal Assumptions**

Assumptions will model valid environmental behavior, including clock and reset conditions and any required restrictions on transaction initiation.

**Formal Verification Engine**

VC Formal will analyze the specified properties and generate pass/fail outcomes or counterexamples, subject to the selected analysis configuration.

**Analysis and Debugging**

The team will use formal logs, counterexample traces, and waveforms to investigate failed properties and refine the verification environment where justified.

### 7.1.4 Verification Strategy

**Formal Property Verification**

FPV will be used to verify whether the RTL satisfies the defined properties under the assumptions of the formal environment.

The primary targets are:

- Functional Unit behavior.
- Master and slave state transitions.
- $I^2C$ protocol conditions.
- Memory access restrictions.
- End-to-end data-consistency properties.

**Sequential Equivalence Checking**

Sequential equivalence checking will be used for suitable modules, such as the D flip-flop, multiplexer, and shift register, when a corresponding reference implementation is available.

**Directed Simulation and Waveform Analysis**

Where simulation is available, directed scenarios may be used to inspect transaction sequences and assist with debugging. Waveform analysis will help correlate assertion failures with signal transitions.

**Incremental Verification**

Verification will proceed from individual blocks to the integrated design. Basic properties will be checked first, followed by protocol-level and end-to-end properties.

This approach helps isolate failures and makes it easier to determine whether they arise from RTL behavior, assertion logic, or the formal environment.

### 7.1.5 Property Development and Formal Stimulus

Unlike a conventional UVM testbench, the primary formal-verification flow explores input sequences according to the RTL, assertions, assumptions, and selected formal-engine configuration.

The initial properties will cover:

- Register behavior and shift-register operations.
- Valid state transitions.
- START and STOP conditions.
- Address and data sequencing.
- ACK/NACK handling.
- Legal and illegal memory accesses.

As verification progresses, additional properties will cover repeated START, multi-byte transactions, clock stretching, and end-to-end consistency.

Assumptions will be reviewed carefully to ensure that they model the intended environment without hiding important failure scenarios.

### 7.1.6 Checking Methodology

**Assertion-Based Checking**

SVA properties will check whether the design satisfies its specified temporal behavior.

Examples include:

- A valid load operation updates the Functional Unit correctly.
- A START condition causes the expected transaction progression.
- A legal memory write updates the addressed location.
- An illegal address does not result in an unintended write.
- A valid read returns the expected stored value.

**Reference-Behavior Checking**

For modules with a suitable reference implementation, formal equivalence checking will compare the reference and implementation according to the tool's configured semantics.

**Counterexample-Based Debugging**

Failed properties will be examined using the generated trace to identify the sequence that violates the expected behavior.

The team will classify failures as potential RTL defects, property-definition issues, environmental-modeling issues, or unresolved cases requiring additional investigation.

**End-to-End Data Checking**

The end-to-end verification goal is to establish that a valid write to a given address is followed by a read of the same address that returns the stored value, subject to the specified reset

behavior and any intervening operations.

The property must account for transaction ordering and intervening writes so that unrelated transactions do not create misleading failures.

**Coverage-Based Checking**

Applicable property coverage and functional coverage will be used to determine whether important behaviors and scenarios have been exercised or analyzed. Passing assertions alone will not be treated as proof that all intended scenarios have been covered.

### 7.1.7 Test-case Scenarios Matrix

The following matrix defines the planned scenarios. Actual pass/fail status will be recorded after the corresponding properties or tests have been implemented and executed.

#### 7.1.7.1 Basic Tests

| Test ID | Test Name | Description |
|---|---|---|
| 1.1.1 | Reset Behavior | Check reset behavior for the modules that support reset. |
| 1.1.2 | Functional Unit Load | Verify that the parallel input is loaded correctly. |
| 1.1.3 | Functional Unit Hold | Verify that the previous value is retained during a hold operation. |
| 1.1.4 | Functional Unit Shift | Check supported left-shift and right-shift operations. |
| 1.1.5 | Basic I²C Write | Verify a valid address and data write transaction. |
| 1.1.6 | Basic I²C Read | Verify that a valid read returns the stored data. |
| 1.1.7 | START/STOP Detection | Check the required protocol conditions. |

#### 7.1.7.2 Protocol and Integration Tests

| Test ID | Test Name | Description |
|---|---|---|
| 1.2.1 | Address Transmission | Verify the address and R/W bit sequence. |
| 1.2.2 | ACK/NACK Handling | Check the response for valid and invalid address scenarios. |
| 1.2.3 | Multi-Byte Transfer | Verify sequencing across multiple data bytes. |
| 1.2.4 | Repeated START | Check a repeated-START transaction sequence. |

| | | |
|---|---|---|
| 1.2.5 | Clock Stretching | Check the specified behavior when the slave holds SCL low. |
| 1.2.6 | End-to-End Data Consistency | Verify that data written to an address is subsequently read back correctly. |
| 1.2.7 | Master–Slave State Coordination | Verify expected transaction progression across both state machines. |

#### 7.1.7.3 Boundary and Negative Tests

| Test ID | Test Name | Description |
|---|---|---|
| 1.3.1 | Lowest Legal Address | Verify access to address 0. |
| 1.3.2 | Highest Legal Address | Verify access to address 127. |
| 1.3.3 | Lowest Illegal Address | Verify handling of address 128. |
| 1.3.4 | Highest Illegal Address | Verify handling of address 255. |
| 1.3.5 | Invalid Address Write | Confirm that an illegal address does not cause an unintended memory update. |
| 1.3.6 | Read Without a Prior Write | Check read behavior against the defined memory initialization and reset model. |
| 1.3.7 | Repeated Transactions | Check successive valid transactions for unexpected state or data corruption. |

# 8 Coverage Requirements

## 8.1.1 Assertions and Property Coverage

The following assertion categories will be developed.

**Functional Unit Assertions**

- Load captures the expected data.
- Hold retains the previous register value.
- Shift and rotate operations follow the specified transformation.
- Serial input bits are incorporated correctly during applicable shifts.

**Master Assertions**

- The master follows the expected transaction sequence.
- START and STOP generation follows the intended protocol behavior.
- Address and data bits are transmitted in the correct order.
- ACK sampling occurs during the expected transaction phase.
- Counter behavior matches the corresponding state progression.

**Slave Assertions**

- START detection initiates address collection.
- Address and R/W information are interpreted correctly.
- ACK/NACK behavior corresponds to the specified address-validity rules.
- Read completion is indicated correctly.
- Illegal addresses do not cause unintended memory writes.
- STOP detection returns the controller to its intended idle behavior.

**Protocol Assertions**

- SDA remains stable while SCL is high, except during valid START and STOP conditions.
- START is detected as an SDA falling transition while SCL is high.
- STOP is detected as an SDA rising transition while SCL is high.
- Data sampling and driving follow the intended timing rules.

**End-to-End Assertions**

- Valid write/read sequences preserve data consistency.
- Read operations do not unintentionally modify stored data.
- Invalid-address transactions do not update memory.
- Transaction sequencing respects the specified interface behavior.

# 9 Project Work Distribution

The project responsibilities will be distributed among Thanmai Neelampalli, Chitra Vagdevi Shivani Kottala, and Bhanu Sai Sidhardha Tummala. Each member will own the verification planning, assertion development, formal analysis, debugging, and documentation for their assigned blocks. Integration, peer review, and final reporting will be collaborative.

1. **Thanmai Neelampalli — Functional Unit, Formal Environment, and Integration Verification**

**Primary responsibilities:**

- Review the Functional Unit architecture and its supporting D flip-flop and 8-to-1 multiplexer implementations.
- Develop assertions for register behavior, reset where applicable, multiplexer selection, and all supported Functional Unit operations: hold, parallel load, logical shifts, rotations, arithmetic shifts, and serial-input handling.
- Perform applicable combinational or sequential equivalence checks when suitable reference implementations are available.
- Establish the common VC Formal environment, including RTL file lists, assertion bindings, clock/reset definitions, and execution scripts, with module-specific inputs from the other members.
- Coordinate integration of the module-level verification environments and develop top-level properties for master–slave state coordination and transaction progression.
- Lead end-to-end write/read consistency checking with the slave/memory verification owner, accounting for address matching, reset behavior, transaction ordering, and intervening writes.
- Maintain the consolidated property-results summary and coordinate regression execution after integration changes.

2. **Chitra Vagdevi Shivani Kottala — $I^2C$ Master and Protocol Verification**

**Primary responsibilities:**

- Review the $I^2C$ Master FSM, signal interfaces, transaction sequence, and bit/burst counters.

- Develop assertions for idle behavior, transaction initiation, START/STOP generation, address and R/W bit transmission, data sequencing, ACK/NACK sampling, and master state transitions.
- Verify master-side transmission and reception control, multi-byte transfers, and repeated-START sequences.
- Develop protocol properties for SDA stability while SCL is high, including legal START/STOP exceptions, and the intended data-driving and sampling behavior.
- Verify the master's response to clock stretching in collaboration with the slave verification owner.
- Define and review master-interface assumptions, execute formal analysis, and investigate counterexamples involving master control or protocol sequencing.
- Document master and protocol property outcomes, relevant coverage evidence, unresolved issues, and representative waveforms.

3. **Bhanu Sai Sidhardha Tummala — I²C Slave/Memory Controller and Memory Verification**

**Primary responsibilities:**

- Review the slave FSM, address decoding, serial-to-parallel conversion, memory interface, and clock-stretching control.
- Develop assertions for START/STOP detection, address and R/W collection, ACK/NACK generation, bit-counter operation, state transitions, read-data availability, and done signaling.
- Verify legal addresses 0–127 and illegal addresses 128–255, including boundary accesses at addresses 0, 127, 128, and 255.
- Verify memory write control, storage at the intended address, correct read-data retrieval, preservation of memory contents during reads, and prevention of unintended writes.
- Check reads without a prior write against the defined memory initialization and reset model.

- Verify slave-side clock stretching through scl_stretch_n and coordinate integrated clock-stretching checks with the master verification owner.
- Support end-to-end readback verification and investigate counterexamples involving address handling, memory updates, or slave transaction sequencing.
- Document slave/memory property outcomes, coverage evidence, unresolved issues, and representative waveforms.

4. Checkpoint-Wise Responsibilities

| **Member** | **Checkpoint 1 — Verification Plan and Initial Implementation** | **Checkpoint 2 — Final Verification, Results, and Documentation** |
|---|---|---|
| Thanmai Neelampalli | Prepare the Functional Unit verification scope; establish the common formal setup; implement initial register, multiplexer, and Functional Unit properties; record preliminary results. | Complete Functional Unit checks and applicable equivalence analysis; coordinate subsystem integration, end-to-end properties, regression execution, and consolidated results. |
| Chitra Vagdevi Shivani Kottala | Review master timing and FSM behavior; prepare master/protocol properties and assumptions; implement initial transaction-sequencing checks. | Complete master and protocol verification, including multi-byte transfers, repeated START, and master response to clock stretching; analyze failures and coverage gaps. |
| Bhanu Sai Sidhardha Tummala | Review slave/memory interfaces and address rules; implement initial decoding, ACK/NACK, and memory-access properties. | Complete slave/memory verification, boundary and negative scenarios, clock stretching, and readback support; analyze failures and coverage gaps. |

**5. Shared Responsibilities**

**All three members will:**

- Review the verification plan, RTL interfaces, assumptions, and property definitions for consistency.

- Review one another's assertions to identify incorrect expectations, excessive constraints, and missing scenarios.
- Classify counterexamples as potential RTL defects, assertion issues, environmental-modeling issues, or unresolved cases.
- Verify repeated transactions and integrated behavior across module boundaries.
- Review property coverage and applicable functional/code coverage, documenting unverified behavior and technical limitations.
- Maintain version-controlled RTL, assertion files, scripts, logs, and verification evidence.
- Contribute their assigned results, challenges, solutions, and waveforms to the final report and presentation.

Each member will retain primary ownership of their assigned deliverables while supporting joint debugging and integration. Verification results will be reported according to the actual tool outcomes and supporting evidence.

## 10 Checkpoint Deliverables and Project Schedule

**Checkpoint 1 — Verification Plan and Initial Implementation (30%)**

**Objective:** Establish the verification strategy and demonstrate initial implementation progress.

**Planned Deliverables**

1. Complete the verification test plan.
2. Review the RTL architecture and signal interfaces.
3. Identify the properties required for the Functional Unit, Master, Slave, and Memory.
4. Organize the relevant RTL files, SVA files, and formal scripts.
5. Develop the initial set of assertions for selected modules.
6. Establish clock/reset definitions and appropriate initial assumptions.
7. Perform preliminary compilation, elaboration, and formal analysis.
8. Record preliminary results and unresolved issues.

**Checkpoint 2 — Final Verification, Results, and Documentation**

**Objective: Expand the properties and increase verification depth** & complete the planned analysis and prepare the final submission.

**Planned Deliverables**

1. Extend SVA properties across all major modules.
2. Add protocol-level checks for START, STOP, ACK/NACK, and data sequencing.
3. Check legal and illegal address handling.
4. Develop properties for multi-byte transfers and repeated START.
5. Verify clock-stretching behavior.
6. Develop end-to-end data-consistency checks.
7. Refine assumptions based on formal counterexamples.
8. Run additional FPV and applicable sequential equivalence checks.
9. Document assertion outcomes, coverage gaps, and debugging progress.
10. Complete the remaining planned assertions and verification activities.
11. Rerun the formal analysis after the final updates.
12. Investigate remaining failures and document their disposition.
13. Review property coverage and applicable functional/code coverage reports.
14. Collect representative formal traces and waveforms.
15. Summarize the verification results and limitations.
16. Document the challenges encountered and the corresponding solutions.
17. Finalize the project report and presentation.
18. Organize the RTL, assertion files, scripts, and verification evidence.